a. Add

b. Subtract

c. Multiply

d. Divide

Select operations form a, b, c, d: "c"

Please enter first number: 11

Please enter second number: 4

11 * 4 = 44

Code: for Simple Calculator

```
def add(P, Q):
  # This function is used for adding two numbers
  return P + Q
def subtract(P, Q):
  # This function is used for subtracting two numbers
  return P - Q
def multiply(P, Q):
  # This function is used for multiplying two numbers
  return P * Q
def divide(P, Q):
  # This function is used for dividing two numbers
  return P / Q
# Now we will take inputs from the user
print ("Please select the operation.")
print ("a. Add")
print ("b. Subtract")
print ("c. Multiply")
print ("d. Divide")

choice = input("Please enter choice (a/ b/ c/ d): ")
```

```
num_1 = int (input ("Please enter the first number: "))
num_2 = int (input ("Please enter the second number: "))

if choice == 'a':
   print (num_1, " + ", num_2, " = ", add(num_1, num_2))

elif choice == 'b':
   print (num_1, " - ", num_2, " = ", subtract(num_1, num_2))

elif choice == 'c':
   print (num1, " * ", num2, " = ", multiply(num1, num2))
elif choice == 'd':
   print (num_1, " / ", num_2, " = ", divide(num_1, num_2))
else:
   print ("This is an invalid input")
```

Output:

Case - (1):

Please select the operation.

a. Add

b. Subtract

c. Multiply

d. Divide

Please enter choice (a/ b/ c/ d): d

Please enter the first number: 1

Please enter the second number: 2

1 / 2 = 0.5

Case - (2):

Please select the operation.

a. Add

b. Subtract

c. Multiply

d. Divide

Please enter choice (a/ b/ c/ d): b

Please enter the first number: 12

Please enter the second number: 11

12 - 11 = 1

Python Program to Display Fibonacci Sequence Using Recursion

Fibonacci sequence:

A Fibonacci sequence is a sequence of integers which first two terms are 0 and 1 and all other terms of the sequence are obtained by adding their preceding two numbers.

For example: 0, 1, 1, 2, 3, 5, 8, 13 and so on...

```
def recur_fibo(n):
  if n <= 1:
     return n
  else:
     return(recur_fibo(n-1) + recur_fibo(n-2))
# take input from the user
nterms = int(input("How many terms? "))
# check if the number of terms is valid
if nterms <= 0:
  print("Plese enter a positive integer")
else:
  print("Fibonacci sequence:")
  for i in range(nterms):
     print(recur_fibo(i))
```

Lecture No. 28

Recursion And Lambda Function.

Lambda Function

In Python, small anonymous (unnamed) functions can be created with lambda keyword. Lambda forms can be used as an argument to other function where function objects are required but syntactically they are restricted to a single expression

A lambda function can take any number of arguments, but can only have one expression.

Syntax

lambda arguments : expression

The expression is executed and the result is returned:

Example

Add 10 to argument a, and return the result:

```
x = lambda a : a + 10
print(x(5))
```

Lambda functions can take any number of arguments:

Example

Multiply argument a with argument b and return the result:

```
x = lambda a, b : a * b
print(x(5, 6))
```

Example

Summarize argument a, b, and c and return the result:

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

Why Use Lambda Functions?

The power of lambda is better shown when you use them as an anonymous function inside another function.

Say you have a function definition that takes one argument, and that argument will be multiplied with an unknown number:

```
def myfunc(n):
  return lambda a : a * n
```

Use that function definition to make a function that always doubles the number you send in:

Example

```
def myfunc(n):
  return lambda a : a * n
  mydoubler = myfunc(2)
print(mydoubler(11))
```

Or, use the same function definition to make a function that always triples the number you send in:

Example

```
def myfunc(n):
  return lambda a : a * n
mytripler = myfunc(3)
print(mytripler(11))
```

Or, use the same function definition to make both functions, in the same program:

Example

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)
```

```
print(mydoubler(11))
print(mytripler(11))
```

Use lambda functions when an anonymous function is required for a short period of time..

A function like this:

```
def average(x, y):
   return (x + y)/2
print(average(4, 3))
```

may also be defined using lambda

```
print((lambda x, y: (x + y)/2)(4, 3))
```

Output:

3.5

Count Down to Zero

The first example is a function called countdown(), which takes a positive number as an argument and prints the numbers from the specified argument down to zero:

```
def countdown(n):
     print(n)
     if n == 0:
        return           # Terminate recursion
    else:
     countdown(n - 1)   # Recursive call
countdown(5)
```

Output

5
4
3
2
1
0

Notice how countdown() fits the paradigm for a recursive algorithm described above:

- The base case occurs when n is zero, at which point recursion stops.
- In the recursive call, the argument is one less than the current value of n, so each recursion moves closer to the base case.

Example

Calculate factorial with function recursion

```
def factorial(n):
    if n <= 1:
        return 1
    else:
        n * factorial(n - 1)

factorial(4)
```

Output

24

Lecture No. 29

Practice of function related programs

Python Program to check Armstrong Number

Suppose we have a given number, "K", and we have to check whether the given number is an Armstrong number or not. In this tutorial, we will learn how to identify the given number is an Armstrong number or not using Python.

Definition:

A positive number of "p" number of digits is called an Armstrong number of order n (order is the number of digits) if:

1. abcde? = pow(a, p) + pow(b, p) + pow(c, p) + pow(d, p) + pow(e, p) + ?

Example:

1. Input: 407
2. Output: Yes, 407 **is** an Armstrong number
3. Solution:
4. (4 * 4 * 4) + (0 * 0 * 0) + (7 * 7 * 7) = 407
5. Input: 1634
6. Output: Yes, 1634 **is** an Armstrong number
7. Solution:
8. (1 * 1 * 1) + (6 * 6 * 6) + (3 * 3 * 3) + (4* 4 * 4) = 1634
9. Input: 253
10. Output: No, 253 **is not** an Armstrong number
11. Solution:
12. (2 * 2 * 2) + (5 * 5 * 5) + (3 * 3 * 3) = 160
13. Input: 1
14. Output: Yes, 1 **is** an Armstrong number
15. Solution:
16. (1* 1) = 1

Example:

```
def power_1(A, B):

    if B == 0:
        return 1
    if B % 2 == 0:
        return power_1(A, B // 2) * power(A, B // 2)

    return A * power(A, B // 2) * power(A, B // 2)

# Function for calculating "order of the number"
def order_1(A):

    # Variable for storing the number
    N = 0
    while (A != 0):
        N = N + 1
        A = A // 10

    return N

# Function for checking if the given number is Armstrong number or not
def is_Armstrong(A):

    N = order_1(A)
    temp_1 = A
    sum_1 = 0

    while (temp_1 != 0):
```

```
    R_1 = temp_1 % 10
    sum_1 = sum_1 + power_1(R_1, N)
    temp_1 = temp_1 // 10

  # If the above condition is satisfied, it will return the result
  return (sum_1 == A)

# Driver code
A = int(input("Please enter the number to be checked: "))
print(is_Armstrong(A))
```

Output:

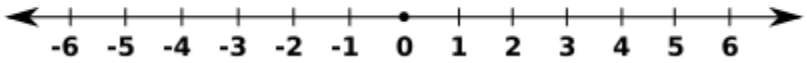
```
1#
Please enter the number to be checked:  0
True
2#
Please enter the number to be checked:  123
False
3#
Please enter the number to be checked:  1634
True
```

Python Program to check if a Number is Positive, Negative or Zero

Here, we will learn to write a Python program through which we can check if the given number is positive, zero, or negative.

Positive Numbers: A number is said to be positive if the number has a value greater than 0, like 1, 2, 3, 5, 7, 9, 11, 13, etc. All the natural numbers are positive numbers.

Negative Numbers: If a given number has a value less than 0 like -1, -2, -3, -5, -7, -9, -11, -13, etc., then we can say that the given number is a negative number. Only rational and integer type numbers can have negative values or numbers.



Let us look at the following example to understand the implementation

Example:

```
# Default function to run if else condition
def NumberCheck(a):
    # Checking if the number is positive
    if a > 0:
        print("Number given by you is Positive")
    # Checking if the number is negative
    elif a < 0:
        print("Number given by you is Negative")
    # Else the number is zero
    else:
        print("Number given by you is zero")
# Taking number from user
a = float(input("Enter a number as input value: "))
# Printing result
NumberCheck(a)
```

Output:

```
Enter a number as input value: -6
Number given by you is Negative
```

Python program to convert Kilometres to Miles

In this tutorial, we will learn a step-by-step approach for how to write a Python program to convert kilometres value into miles value.

Understanding the Logic Behind the Conversion of Kilometre to Miles

Let's begin with the basics; first, we will discuss the measuring units. As we know, Kilometre and Miles represent the unit of length.

1 kilometre equals 0.62137 miles.

Miles = kilometre * 0.62137

And,

Kilometre = Miles / 0.62137

Hence, the value "0.62137" would be considered as the factor of conversation or the ratio for converting units.

Steps to be Followed for Converting Kilometre unit to Miles Unit.

We follow the below steps for getting a clear idea of how to convert kilometres to miles units.

Step 1: We will define the variable for storing the value of kilometre_1 and accepting the input from the user.

```
1. kilometre_1 = float (input ("Please enter the speed of car in Kilometre as a unit: "))
```

Step 2: Now, we will define and store the conversion factor into the variable.

```
1. conversion_ratio_1 = 0.621371
```

Step 3: Then, we will define the variable for storing the value of kilometre_1 which is converted into miles. Then in addition we will write the logic of converting kilometre to miles units.

```
1. miles_1 = kilometre_1 * conversion_ratio_1
```

Step 4: At last, we will display the converted value by using the print() function.

```
1. print ("The speed value of car in Miles: ", miles_1)
```

Full Extension of Code:

```
1. kilometre_1 = float (input ("Please enter the speed of car in Kilometre as a unit: "))
2. conversion_ratio_1 = 0.621371
3. miles_1 = kilometre_1 * conversion_ratio_1
```

4. **print** ("The speed value of car in Miles: ", miles_1)

Output:

Please enter the speed of car in Kilometre as a unit: 16

The speed value of car in Miles: 9.941936

Approach 2:

We can also use another approach of simply statistically defining a function for converting kilometres to miles unit.

Example:

```
1. def kilometre_1(km):
2.     conversion_ratio_1= 0.621371
3.     miles_1 = km * conversion_ratio_1
4.     print ("The speed value of car in Miles: ", miles_1)
5. km = float (input ("Please enter the speed of car in Kilometre as a unit: "))
6. kilometre_1(km)
```

Output:

Please enter the speed of car in Kilometre as a unit: 14

The speed value of car in Miles: 8.699194

Lecture No. 30

Files: Open, read, write, append and close. Tell and seek methods

Python File Open

File handling is an important part of any web application.

Python has several functions for creating, reading, updating, and deleting file

File Handling

The key function for working with files in Python is the open() function.

The open() function takes two parameters; *filename*, and *mode*.

There are four different methods (modes) for opening a file:

"r" - Read - Default value. Opens a file for reading, error if the file does not exist

"a" - Append - Opens a file for appending, creates the file if it does not exist

"w" - Write - Opens a file for writing, creates the file if it does not exist

"x" - Create - Creates the specified file, returns an error if the file exists

In addition you can specify if the file should be handled as binary or text mode

"t" - Text - Default value. Text mode

"b" - Binary - Binary mode (e.g. images)

Syntax

To open a file for reading it is enough to specify the name of the file:

```
f = open("demofile.txt")
```

The code above is the same as:

```
f = open("demofile.txt", "rt")
```

Because "r" for read, and "t" for text are the default values, you do not need to specify them.

Note: Make sure the file exists, or else you will get an error.

Python File Open

Assume we have the following file, located in the same folder as Python:

demofile.txt

Hello! Welcome to demofile.txt

This file is for testing purposes.

Good Luck!

To open the file, use the built-in open() function.

The open() function returns a file object, which has a read() method for reading the content of the file:

Example

```
f = open("demofile.txt", "r")
print(f.read())
```

If the file is located in a different location, you will have to specify the file path, like this:

Example

Open a file on a different location:

```
f = open("D:\\myfiles\welcome.txt", "r")
print(f.read())
```

Read Only Parts of the File

By default the read() method returns the whole text, but you can also specify how many characters you want to return:

Example

Return the 5 first characters of the file:

```
f = open("demofile.txt", "r")
```

Write to an Existing File

To write to an existing file, you must add a parameter to the open() function:

"a" - Append - will append to the end of the file

"w" - Write - will overwrite any existing content

Example

Open the file "demofile2.txt" and append content to the file:

```
f = open("demofile2.txt", "a")
f.write("Now the file has more content!")
f.close()

#open and read the file after the appending:
f = open("demofile2.txt", "r")
print(f.read())
```

Example

Open the file "demofile3.txt" and overwrite the content:

```
f = open("demofile3.txt", "w")
f.write("Woops! I have deleted the content!")
f.close()

#open and read the file after the overwriting:
f = open("demofile3.txt", "r")
print(f.read())
```

Note: the "w" method will overwrite the entire file.

Create a New File

To create a new file in Python, use the open() method, with one of the following parameters:

"x" - Create - will create a file, returns an error if the file exist

"a" - Append - will create a file if the specified file does not exist

"w" - Write - will create a file if the specified file does not exist

Example

Create a file called "myfile.txt":

```
f = open("myfile.txt", "x")
```

Result: a new empty file is created!

Example

Create a new file if it does not exist:

```
f = open("myfile.txt", "w")
```

Delete a File

To delete a file, you must import the OS module, and run its os.remove() function:

Example

Remove the file "demofile.txt":

```
import os
os.remove("demofile.txt")
```

Check if File exist:

To avoid getting an error, you might want to check if the file exists before you try to delete it:

Example

Check if file exists, *then* delete it:

```
import os
if os.path.exists("demofile.txt"):
  os.remove("demofile.txt")
else:
  print("The file does not exist")
```

Delete Folder

To delete an entire folder, use the os.rmdir() method:

Example

Remove the folder "myfolder":

```
import os
os.rmdir("myfolder")
```

Note: You can only remove *empty* folders.

Lecture No. 31

Practice of file related programs

Create File in Python

Python is widely used in data analytics and comes with some inbuilt functions to work with files. We can create a file and do different operations, such as write a file and read a file using Python.

After reading this tutorial, you'll learn: –

- Create a file in the current directory or a specified directory
- Create a file if not exists
- Create a file with a date and time as its name
- Create a file with permissions

Table of contents

- Create A Empty Text File
- Create File In A Specific Directory
- Create a File If Not Exists
- Create File with a DateTime
- Create a file with Permission

Create A Empty Text File

We don't have to import any module to create a new file. We can create a file using the built-in function open().

open('file_Path', 'access_mode')

Pass the file name and access mode to the open() function to create a file. Access mode specifies the purpose of opening a file.

Below is the list of access modes for creating an a file.

File Mode	Meaning
w	Create a new file for writing. If a file already exists, it truncates the file first. Use to create and write content into a new file.

File Mode	Meaning
x	Open a file only for exclusive creation. If the file already exists, this operation fails.
a	Open a file in the append mode and add new content at the end of the file.
b	Create a binary file
t	Create and open a file in a text mode

File access mode

Example: **Create a new empty text file** named 'sales.txt'

```
# create a empty text file
# in current directory
fp = open('sales.txt', 'x')
fp.close()
```

Use **access mode w** if you want to create and write content into a file.

```
# create a empty text file
fp = open('sales_2.txt', 'w')
fp.write('first line')
fp.close()
```

As you can see in the image two new files gets created in the account folder.

created files

Note:

- The file is created in the same directory where our program/script is running.
- If you have not specified any specific path(directory location), the file is created in the working directory. It is known as creating a file using the **relative path**. A relative path contains the current directory and then the file name.

You can **verify the result** using the following four approaches

1. If the script executed without an error or exception
2. By checking the working directory manually to look for a new file
3. Use the os.listdir(directory_path) function to list all files from a folder before and after creating a file

4. Use the os.path.isfile(file_path) function to verify if a newly created file exists in a directory.

Let's verify our operation result.

```
import os

# list files from a working directory
print(os.listdir())

# verify file exist
print(os.path.isfile('sales.txt'))
```

Output

```
['sample.txt', 'sales.txt', 'sales_2.txt']
True
```

Create File In A Specific Directory

To create a file inside a specific directory, we need to open a file using the absolute path. An absolute path contains the entire path to the file or directory that we need to use.

It includes the **complete directory list required to locate the file**. For example, /user/Pynative/data/sales.txt is an absolute path to discover the sales.txt. All of the information needed to find the file is contained in the path string.

Let's see the example to **create a file for writing** using the absolute path.

```
# create a text file for writing
with open(r'E:\pynative\reports\profit.txt', 'w') as fp:
    fp.write('This is first line')
    pass
```

Note: Using the with statement a **file is closed automatically** it ensures that all the resources that are tied up with the file are released.

Let's verify result using the absolute path.

```
import os
```

```
# list files a directory
print(os.listdir(r'E:\pynative\reports'))
# output ['sample.txt', 'sales.txt', 'sales_2.txt' 'profit.txt']

# verify file exist
print(os.path.isfile(r'E:\pynative\reports\profit.txt'))
# output True
```

Also, you can **join directory path and file name** to create file at the specified location.

If you have a directory path and file name in two variables, use the os.path.join() function to construct a full path. This function accepts the directory path and file name as arguments and constructs an absolute path to create a file.

Example:

```
import os

# Specify the directory path
path = r'E:\pynative\account'
file_name = 'revenue.txt'

# Creating a file at specified folder
# join directory and file path
with open(os.path.join(path, file_name), 'w') as fp:
    # uncomment below line if you want to create an empty file
    fp.write('This is a new line')
```

Create a File If Not Exists

Sometimes it is essential not to create a new file if a file with the same name already exists in a given path. By default, when you open a file in write mode, it overwrites it if it exists. Else, create the new one.

We can create a file only if it is not present using the following two ways:

- Use os.path.exists("file_path") function to check if a file exists.

- Use the access mode x in the open() function and exception handling.

Example 1: create file if not exists.

```
import os


file_path = r'E:\pynative\account\profit.txt'
if os.path.exists(file_path):
    print('file already exists')
else:
    # create a file
    with open(file_path, 'w') as fp:
        # uncomment if you want empty file
        fp.write('This is first line')
```

Example 2: Use file access mode x

The access mode x open a file for exclusive creation. If the file already exists, this operation fails with FileExistsError. Use try-except block to handle this error.

```
try:
    file_path = r'E:\pynative\account\profit.txt'
    # create file
    with open(file_path, 'x') as fp:
        pass
except:
    print('File already exists')
```

Create File with a DateTime

Let's see how to create a text file with the current date as its name. Use the datetime module to get the current date and time and assign it to the file name to create a file with the date and time in its name.

- Python provides a datetime module that has several classes to access and manipulate the date and timestamp value.
- First, get the current datetime value

- Next, we need to format datetime into a string to use it as a file name.
- At last, pass it to the open() function to create a file

Example

```
from datetime import datetime

# get current date and time
x = datetime.now()

# create a file with date as a name day-month-year
file_name = x.strftime('%d-%m-%Y.txt')
with open(file_name, 'w') as fp:
    print('created', file_name)

# with name as day-month-year-hours-minutes-seconds
file_name_2 = x.strftime('%d-%m-%Y-%H-%M-%S.txt')
with open(file_name_2, 'w') as fp:
    print('created', file_name_2)

# at specified directory
file_name_3 = r"E:\demos\files_demos\account\\" + x.strftime('%d-%m-%Y-%H-%M-%S.txt')
with open(file_name_3, 'w') as fp:
    print('created', file_name_3)
```

Output:

```
created 29-06-2021.txt
created 29-06-2021-14-57-24.txt
created E:\demos\files_demos\account\\29-06-2021-14-57-24.txt
```

Create a file with Permission

Let's see how to create a file with permissions other users can write.

- To create a file with appropriate permissions, use os.open() to create the file descriptor and set the permission.
- Next, open the descriptor using the built-in function open()

```
import os

file_path = r'E:\pynative\account\sample.txt'
# The default umask is 0o22 which turns off write permission of group and others
os.umask(0)
with open(os.open(file_path, os.O_CREAT | os.O_WRONLY, 0o777), 'w') as fh:
    fh.write('content')
```

Lecture No. 32

Practice of file related programs

Python File Seek(): Move File Pointer Position

Python offers several methods for file handling. In addition to the standard operations like reading and writing to the files, there are methods to manipulate the file pointer effectively.

In this tutorial, you'll learn how to use the seek() function to **move the position of a file pointer** while reading or writing a file.

Table of contents

- What is seek() in Python
- How to Use seek() Method
  - Example
- Seek the Beginning of the File
- Seeking The End of File
- Seek From The Current Position
- Seek backward With Negative Offset
- tell() Function To Get File Handle Position
- Summary

What is seek() in Python

The seek() function **sets the position of a file pointer** and the tell() function **returns the current position** of a file pointer.

A **file handle or pointer denotes the position from which the file contents will be read or written**. File handle is also called as file pointer or cursor.

For example, when you open a file in write mode, the file pointer is placed at the 0th position, i.e., at the start of the file. However, it changes (increments) its position as you started writing content into it.

Or, when you read a file line by line, the file pointer moves one line at a time.

Sometimes we may have to read only a specific portion of the file, in such cases use the seek() method to move the file pointer to that position.

For example, use the seek() function to do the file operations like: –

- Read a file from the 10th character.
- Directly jump to the 5th character from the end of the file.
- Add new content to file after a particular position.

How to Use seek() Method

To change the file handle's position use seek() method. As we discussed, the seek() method sets the file's current position, and then we can read or write to the file from that position.

Syntax:

f.seek(offset, whence)

How many points the pointer will move is **computed from adding offset to a reference point**; the reference point is given by the whence argument.

The allowed values for the whence argument are: –

- A whence value of **0** means from the beginning of the file.
- A whence value of **1** uses the current file position
- A whence value of **2** uses the end of the file as the reference point.

The default value for the whence is the beginning of the file, which is **0**

Refer to the below **table for clear understanding**.

Seek Operation	Meaning
f.seek(0)	Move file pointer to the beginning of a File
f.seek(5)	Move file pointer five characters ahead from the beginning of a file.
f.seek(0, 2)	Move file pointer to the end of a File
f.seek(5, 1)	Move file pointer five characters ahead from the current position.
f.seek(-5, 1)	Move file pointer five characters behind from the current position.
f.seek(-5, 2)	Move file pointer in the reverse direction. Move it to the 5th character from the end

Seek Operation	Meaning
	of the file

File seek function

Example

Consider the following example where we are reading a text file contents with the **offset as 6**. It means we will start **reading the file directly from the 6th character**.

text file

```
with open(r'E:\demos\files_demos\sample.txt', "r") as fp:
    # Moving the file handle to 6th character
    fp.seek(6)
    # read file
    print(fp.read())
```

Output

```
line
Second line
Third line
Fourth line
Fifth line
Sixth line
Seventh line
Eighth line
```

As you can see in the output, the first six characters are missing.

Seek the Beginning of the File

We can move the file pointer to the beginning of the file using the seek() method by passing the setting whence to **0**.

The 0 indicates the first byte, which is the beginning of the file.

Example

Let's see how to bring the file cursor to the beginning of the file.

In this example, we are writing to the text file. After adding content, we wanted to move the pointer to the beginning of the file to read the entire file.

```
# open file in write and read mode w+
with open(r'E:\demos\files_demos\test.txt', "w+") as fp:
    # add content
    fp.write('My First Line\n')
    fp.write('My Second Line')
    # move pointer to the beginning
    fp.seek(0)
    # read file
    print(fp.read())
```

Output

```
My First Line
My Second Line
```

Seeking The End of File

Set whence to **2** and the **offset to 0** to move the file pointer to the end of the file.

- In the below example, we will perform the following three operations
- We will move the file pointer at the end of the file and write new content
- Next, we will move the file pointer at the start of the file and write fresh content at the beginning of the file.
- Again, we will move the file pointer to the end of the file and write more content.

Example:

Let's see how to move the file cursor to the end of the file. We will use the existing file for this operation and open a file in read and write mode.

```
# open file for reading and writing  a+
with open(r'E:\demos\files_demos\test.txt', "r+") as fp:
    # Moving the file handle to the end of the file
    fp.seek(0, 2)
```

```
# Inserting new content to the end of the file
fp.write("\nThis content is added to the end of the file")

# moving to the beginning
# again read the whole file
fp.seek(0)
print(fp.read())
```

Output

My First Line

My Second Line

This content is added to the end of the file

Seek From The Current Position

We can move the file pointer few positions ahead from the current position by setting the whence to 1 and offset to the number of the position you want to move.

For example, the current file pointer is at 20th position, and you wanted to jump to the 75th character then set offset to 50 and whence to 1.

We will use the existing file for this operation and open a file in read and write mode.

Note:

If your using seek() function in text files (those opened without a b in the access mode), only seeks relative to the beginning of the file are allowed.

If you try to move the file handle from the current position you'll get an io.UnsupportedOperation: can't do nonzero cur-relative seeks error.

So **open the file in binary mode if you want to move the file pointer ahead or behind** from the current position

Example: Move the file handle 10 points ahead from current position.

Note:

- **Open file in binary mode.** For reading use the rb, for writing use the wb, and for both reading and writing use rb+.
- **Convert byte to string** if you are reading a text file.

```
# Open file for reading in Binary mode
with open(r'E:\demos\files_demos\test.txt', "rb") as fp:
    # Move the file handle to the 5th character
    # from the beginning of the file
    fp.seek(3)
    # read 5 bytes and convert it to string
    print(fp.read(5).decode("utf-8"))

    # Move the fp 10 points ahead from current position
    # read 5 bytes and convert it to string
    fp.seek(10, 1)
    # again read 6 bytes
    print(fp.read(6).decode("utf-8"))
```

Seek backward With Negative Offset

In some cases, we have to read characters from the end of the file. to do that, we need to move the file pointer in a reverse direction.

Here, you'll learn how to **seek file handle backward from current position as well as from the end of file**.

For example, **move to the 10th character from the end of the file**. This can be done by setting the offset corresponding to the end of the file.

Example:

In this example, we are opening the file in the read binary mode (rb) and pass the offset corresponding to the end of the file.

```
# Open file for reading in binary mode
with open(r'E:\demos\files_demos\test.txt', "rb") as fp:
    # Move in reverse direction
    # move to the 40th character from the end of the file
    fp.seek(-40, 2)
    # read 11 bytes and convert it to string
```

```
    print(fp.read(11).decode("utf-8"))
```

Output

content is

Use the below code **to Seek backwards from current position**

```
# Open file for reading in binary mode
with open(r'E:\demos\files_demos\test.txt', "rb") as fp:
    # read first 8 bytes
    print(fp.read(8).decode('utf-8'))
    # Move in reverse direction
    # move to the 5th behind from current position
    fp.seek(-5, 1)
    # read 10 bytes and convert it to string
    print(fp.read(10).decode("utf-8"))
```

Output:

My First

First Line

tell() Function To Get File Handle Position

While the file's access mode implies the type of operation that we intend to perform in the file, it also determines the filehandle position. For example, if the file is opened in reading, the file handle will be in the beginning, and after reading the entire file, it will be in the last character, which is the End of the File.

We can get the file handle current position using the tell() method.

Syntax:

file_object.tell()

There are no arguments for this method. The **return value is the integer** representing the file handle position.

```
# open file for reading and writing  r+
with open(r'E:\demos\files_demos\test.txt', "r+") as fp:
    # Moving the file handle to the end of the file
```

```
fp.seek(0, 2)
# getting the file handle position
print('file handle at:', fp.tell())
# writing new content
fp.write("\nDemonstrating tell")
# getting the file handle position
print('file handle at:', fp.tell())
# move to the beginning
fp.seek(0)
# getting the file handle position
print('file handle at:', fp.tell())
# read entire file
print('***Printing File Content***')
print(fp.read())
print('***Done***')
# getting the file handle position
print('file handle at:', fp.tell())
```

Output

```
file handle at: 75
file handle at: 95
file handle at: 0
***Printing File Content***
My First Line
My Second Line
This content is added to the end of the file
Demonstrating tell
***Done***
file handle at: 95
```

MITRC
MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Chakravarty Sharma

Subject & Subject Code: Introduction to Python Programming & 4CS4-05

Semester: IV **Session:** 2022-23

Unit No.: 5 **Date of Submission:** 31/03/2023

Lecture No.	Topics Covered	Page No.	Total Page
33	**NumPy Basics:** Arrays and Vectorized Computation	2- 6	5
34	The NumPy NM Array – Creation ND arrays- Data Types For ND arrays	7-11	5
35	Arithmetic with NumPy arrays- basic indexing and slicing- Boolean indexing- transposing array and Swapping Axes.	12-18	7
36	Universal Function: Fast Element-Wise array Function	19-23	5
37	NumPy related Examples	24-29	6
38	Arithmetic with NumPy arrays- basic indexing and slicing- Boolean indexing- transposing array and Swapping Axes.	30-33	4
39	Mathematical and statistical methods- Sorting Unique and Other set Logic	34-40	7
40	Data Visualization	41-46	6

References:

1. Programming Python ny Mark Lutz, O`Reilly
2. Intrduction to Programming using Python by Y.Daniel Liang, Pearson, 2012

Faculty **HOD** **Dean Academics**

Lecture No. 33

NumPy Basics: Arrays and Vectorized Computation

What is NumPy?
NumPy is a Python library used for working with arrays.
It also has functions for working in domain of linear algebra, fourier transform, and matrices.
It is an open source project and you can use it freely.
NumPy stands for Numerical Python.

Why Use NumPy?
In Python we have lists that serve the purpose of arrays, but they are slow to process.
NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.
Arrays are very frequently used in data science, where speed and resources are very important.

Data Science: is a branch of computer science where we study how to store, use and analyze data for deriving information from it.

Why is NumPy Faster Than Lists?
NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.
This behavior is called locality of reference in computer science.
This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.

Which Language is NumPy written in?
NumPy is a Python library and is written partially in Python, but most of the parts that require fast computation are written in C or C++.

Installation of NumPy
If you have Python and PIP already installed on a system, then installation of NumPy is very easy.
Install it using this command:
C:\Users*Your Name*>pip install numpy
If this command fails, then use a python distribution that already has NumPy installed like, Anaconda, Spyder etc.

Import NumPy
Once NumPy is installed, import it in your applications by adding the import keyword:
import numpy

Now NumPy is imported and ready to use.

Example
```
import numpy
arr = numpy.array([1, 2, 3, 4, 5])
print(arr)
```

NumPy as np
NumPy is usually imported under the np alias.

alias: In Python alias are an alternate name for referring to the same thing.

Create an alias with the as keyword while importing:
import numpy as np
Now the NumPy package can be referred to as np instead of numpy.
Example
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

Checking NumPy Version
The version string is stored under __version__ attribute.

Example
```
import numpy as np
print(np.__version__)
```

What is Vectorization?
Vectorization is a technique to implement arrays without the use of loops. Using a function instead can help in minimizing the running time and execution time of code efficiently. Various operations are being performed over vector instead of arrays such as dot product of vectors which is also known as scalar product as it produces single output, outer products which results in square matrix of dimension equal to (length X length) of the vectors, Element wise multiplication which products the element of same indexes and dimension of the matrix remain unchanged.

Vectorization is the process of converting an algorithm from operating on a single value at a time to operating on a set of values (vector) at one time.

Hence, we can use these techniques to perform operations on Numpy arrays without using loops. It only uses the pre-defined inbuilt functions like the sum function etc. and mathematical operations like "+", "-", "/" etc. for performing operations in Numpy arrays.

Vectorization also uses the concept of broadcasting for performing operations in arrays of different sizes (discussed later in this article).

Let's look at the following examples using np vectorize to understand the concept of vectorization:

```
import numpy as np
arr=np.arange(1,20,4)
arr1=np.arange(1,10,2)
print("Array arr: ",arr)
print("Array arr1: ",arr1)
arr2=(arr+arr1)
print(" The sum of elements of arr and arr1 using vectorization: ",arr2)
```

Output:
Array arr: [1 5 9 13 17]
Array arr1: [1 3 5 7 9]
The sum of elements of arr and arr1 using vectorization: [2 8 14 20 26]

Explanation: Two arrays, arr, and arr1, are created using the arange function. These both contain the same number of elements. The addition between the elements of an array is performed using the mathematical operator "+". The addition is in an element-by-element manner, like the first element of arr is added to the first element of arr1 and so on.

We can also perform vectorization using the np.vectorize function. Let's see how to use the np vectorize function.

Numpy Vectorization with the numpy.vectorize() Function

The NumPy vectorize function (np.vectorize) is provided by the Python library. It accepts a nested sequence of objects or a NumPy array as input and returns a single NumPy array or a tuple of NumPy arrays as output. With the addition of the NumPy broadcasting rules, the np vectorize function evaluates pyfunc(user-defined function parameter) over successive tuples of the input arrays, similar to the map function in Python.

Vectorized Implementation

```
import numpy as np
arr=np.arange(1,20,4)
sum=0
arr2=np.sum(arr)
```

```
print("Sum of array elements using vectorization: ",arr2)
```

Output:

Sum of array elements using vectorization: 45

Explanation: If we use the inbuilt sum function on array arr in place of the for loop, we get the same output as 45. This is vectorization.

```
import numpy as np
arr=np.arange(1,20,4)
arr1=np.arange(1,10,2)
print("Array arr: ",arr)
print("Array arr1: ",arr1)
arr2=np.dot(arr,arr1)
print("Product of elements of arr and arr1 using vectorization: ",arr2)
```

Output:

Array arr: [1 5 9 13 17]
Array arr1: [1 3 5 7 9]
Product of elements of arr and arr1 using vectorization: 305

Explanation: In this example, two arrays, arr, and arr1, are created using the arange function. The inbuilt function dot is used for performing dot products between the elements of an array.

Non-Vectorized Implementation

```
import numpy as np
arr=np.arange(1,20,4)
sum=0
for i in arr:
  sum=sum+i
print("Sum of array elements using for loop: ",sum)
```

Output:

Sum of array elements using for loop: 45

Explanation: In the above code example, an array is created using the arange function. Initiating the sum variable with zero. The elements of the array are added using a for loop, and the updated sum is stored in the sum variable. After printing the sum, we get 45 as an output.

Syntax of NumPy Vectorize

numpy.vectorize(pyfunc, otypes=None, doc=None, excluded=None, cache=False, signature=None)

In the above syntax, numpy.vectorize is a function class with different parameters. Let's quickly understand the use of the parameters of the np vectorize function:

Required Parameters

- **pyfunc:** pyfunc is a required function or method. It is a parameter that can be called or defined by the user.

Iterating Arrays

Iterating means going through elements one by one.
As we deal with multi-dimensional arrays in numpy, we can do this using basic for loop of python.
If we iterate on a 1-D array it will go through each element one by one.

Example

Iterate on the elements of the following 1-D array:

```
import numpy as np
arr = np.array([1, 2, 3])
for x in arr:
  print(x)
```

Iterating 2-D Arrays

In a 2-D array it will go through all the rows.
Example
Iterate on the elements of the following 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
  print(x)
```

If we iterate on a *n*-D array it will go through n-1th dimension one by one.
To return the actual values, the scalars, we have to iterate the arrays in each dimension.
Example
Iterate on each scalar element of the 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
  for y in x:
    print(y)
```

Lecture No. 34

The NumPy NM Array – Creation ND arrays- Data Types For ND arrays

Iterating 3-D Arrays
In a 3-D array it will go through all the 2-D arrays.
Example
Iterate on the elements of the following 3-D array:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
for x in arr:
  print(x)
```

To return the actual values, the scalars, we have to iterate the arrays in each dimension.

Example
Iterate down to the scalars:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

for x in arr:
  for y in x:
    for z in y:
      print(z)
```

Iterating Arrays Using nditer()
The function nditer() is a helping function that can be used from very basic to very advanced iterations. It solves some basic issues which we face in iteration, lets go through it with

Examples.
Iterating on Each Scalar Element
In basic for loops, iterating through each scalar of an array we need to use *n* for loops which can be difficult to write for arrays with very high dimensionality.

Example
Iterate through the following 3-D array:

```
import numpy as np
arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
for x in np.nditer(arr):
  print(x)
```

Iterating Array With Different Data Types
We can use op_dtypes argument and pass it the expected datatype to change the datatype of elements while iterating.

NumPy does not change the data type of the element in-place (where the element is in array) so it needs some other space to perform this action, that extra space is called buffer, and in order to enable it in nditer() we pass flags=['buffered'].

Example
Iterate through the array as a string:

```
import numpy as np
arr = np.array([1, 2, 3])
for x in np.nditer(arr, flags=['buffered'], op_dtypes=['S']):
  print(x)
```

Iterating With Different Step Size
We can use filtering and followed by iteration.

Example
Iterate through every scalar element of the 2D array skipping 1 element:

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
for x in np.nditer(arr[:, ::2]):
  print(x)
```

Enumerated Iteration Using ndenumerate()
Enumeration means mentioning sequence number of somethings one by one.
Sometimes we require corresponding index of the element while iterating,
the ndenumerate() method can be used for those usecases.

Example
Enumerate on following 1D arrays elements:

```
import numpy as np

arr = np.array([1, 2, 3])
for idx, x in np.ndenumerate(arr):
  print(idx, x)
```

Example
Enumerate on following 2D array's elements:

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
```

```
for idx, x in np.ndenumerate(arr):
  print(idx, x)
```

Create a NumPy ndarray Object
NumPy is used to work with arrays. The array object in NumPy is called ndarray.
We can create a NumPy ndarray object by using the array() function.
ExampleGet your own Python Server

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))
```

type(): This built-in Python function tells us the type of the object passed to it. Like in above code it shows that arr is numpy.ndarray type.
To create an ndarray, we can pass a list, tuple or any array-like object into the array() method, and it will be converted into an ndarray:
Example

Use a tuple to create a NumPy array:

```
import numpy as np
arr = np.array((1, 2, 3, 4, 5))
print(arr)
```

Dimensions in Arrays
A dimension in arrays is one level of array depth (nested arrays).
nested array: are arrays that have arrays as their elements.

0-D Arrays
0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

Example
Create a 0-D array with value 42

```
import numpy as np

arr = np.array(42)

print(arr)
```

1-D Arrays
An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.
These are the most common and basic arrays.

Example

Create a 1-D array containing the values 1,2,3,4,5:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

2-D Arrays

An array that has 1-D arrays as its elements is called a 2-D array.
These are often used to represent matrix or 2nd order tensors.
NumPy has a whole sub module dedicated towards matrix operations called numpy.mat

Example

Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr)
```

3-D arrays

An array that has 2-D arrays (matrices) as its elements is called 3-D array.
These are often used to represent a 3rd order tensor.

Example

Create a 3-D array with two 2-D arrays, both containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print(arr)
```

Check Number of Dimensions?

NumPy Arrays provides the ndim attribute that returns an integer that tells us how many dimensions the array have.

Example

Check how many dimensions the arrays have:

```
import numpy as np
a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(a.ndim)
print(b.ndim)
```

```
print(c.ndim)
print(d.ndim)
```

Higher Dimensional Arrays

An array can have any number of dimensions.
When the array is created, you can define the number of dimensions by using the ndmin argument.

Example

Create an array with 5 dimensions and verify that it has 5 dimensions:

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('number of dimensions :', arr.ndim)
```

Lecture No. 35

Arithmetic with NumPy arrays- basic indexing and slicing- Boolean indexing- transposing array and swapping Axes.

Access Array Elements
Array indexing is the same as accessing an array element.
You can access an array element by referring to its index number.
The indexes in NumPy arrays start with 0, meaning that the first element has index 0, and the second has index 1 etc.

Example
Get the first element from the following array:

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[0])
```

Example
Get the second element from the following array.

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[1])
```

Example
Get third and fourth elements from the following array and add them.

```
import numpy as np

arr = np.array([1, 2, 3, 4])
print(arr[2] + arr[3])
```

Access 2-D Arrays
To access elements from 2-D arrays we can use comma separated integers representing the dimension and the index of the element.

Think of 2-D arrays like a table with rows and columns, where the dimension represents the row and the index represents the column.

Example
Access the element on the first row, second column:

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('2nd element on 1st row: ', arr[0, 1])
```

Example
Access the element on the 2nd row, 5th column:

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('5th element on 2nd row: ', arr[1, 4])
```

Access 3-D Arrays
To access elements from 3-D arrays we can use comma separated integers representing the dimensions and the index of the element.

Example
Access the third element of the second array of the first array:

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
print(arr[0, 1, 2])
```

Example
arr[0, 1, 2] prints the value 6.
And this is why:

The first number represents the first dimension, which contains two arrays:
[[1, 2, 3], [4, 5, 6]]
and:
[[7, 8, 9], [10, 11, 12]]
Since we selected 0, we are left with the first array:
[[1, 2, 3], [4, 5, 6]]
The second number represents the second dimension, which also contains two arrays:
[1, 2, 3]
and:
[4, 5, 6]
Since we selected 1, we are left with the second array:
[4, 5, 6]
The third number represents the third dimension, which contains three values:
4
5
6
Since we selected 2, we end up with the third value:
6

Negative Indexing
Use negative indexing to access an array from the end.

Example
Print the last element from the 2nd dim:

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('Last element from 2nd dim: ', arr[1, -1])
```

NumPy Array Slicing

Slicing arrays
Slicing in python means taking elements from one given index to another given index.
We pass slice instead of index like this: [*start*:*end*].
We can also define the step, like this: [*start*:*end*:*step*].
If we don't pass start its considered 0
If we don't pass end its considered length of array in that dimension
If we don't pass step its considered 1

Example
Slice elements from index 1 to index 5 from the following array:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])
```

Note: The result *includes* the start index, but *excludes* the end index.
Example
Slice elements from index 4 to the end of the array:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[4:])
```

Example
Slice elements from the beginning to index 4 (not included):

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[:4])
```

Negative Slicing
Use the minus operator to refer to an index from the end:

Example
Slice from the index 3 from the end to index 1 from the end:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[-3:-1])
```

STEP
Use the step value to determine the step of the slicing:

Example
Return every other element from index 1 to index 5:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5:2])
```

Example
Return every other element from the entire array:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[::2])
```

Slicing 2-D Arrays
Example
From the second element, slice elements from index 1 to index 4 (not included):

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[1, 1:4])
```

Note: Remember that *second element* has index 1.

Example
From both elements, return index 2:

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 2])
```

Example
From both elements, slice index 1 to index 4 (not included), this will return a 2-D array:

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 1:4])
```

NumPy Data Types

Data Types in Python
By default Python have these data types:

- strings - used to represent text data, the text is given under quote marks. e.g. "ABCD"
- integer - used to represent integer numbers. e.g. -1, -2, -3
- float - used to represent real numbers. e.g. 1.2, 42.42
- boolean - used to represent True or False.
- complex - used to represent complex numbers. e.g. 1.0 + 2.0j, 1.5 + 2.5j

Data Types in NumPy
NumPy has some extra data types, and refer to data types with one character, like i for integers, u for unsigned integers etc.
Below is a list of all data types in NumPy and the characters used to represent them.

- i - integer
- b - boolean
- u - unsigned integer
- f - float
- c - complex float
- m - timedelta
- M - datetime
- O - object
- S - string
- U - unicode string
- V - fixed chunk of memory for other type (void)

Checking the Data Type of an Array
The NumPy array object has a property called dtype that returns the data type of the array:
Example

Get the data type of an array object:

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr.dtype)
```

Example
Get the data type of an array containing strings:

```
import numpy as np
arr = np.array(['apple', 'banana', 'cherry'])
print(arr.dtype)
```

Creating Arrays With a Defined Data Type

We use the array() function to create arrays, this function can take an optional argument: dtype that allows us to define the expected data type of the array elements:

Example

Create an array with data type string:

```
import numpy as np
arr = np.array([1, 2, 3, 4], dtype='S')
print(arr)
print(arr.dtype)
```

For i, u, f, S and U we can define size as well.

Example

Create an array with data type 4 bytes integer:

```
import numpy as np
arr = np.array([1, 2, 3, 4], dtype='i4')
print(arr)
print(arr.dtype)
```

Example

A non integer string like 'a' can not be converted to integer (will raise an error):

```
import numpy as np
arr = np.array(['a', '2', '3'], dtype='i')
```

Converting Data Type on Existing Arrays

The best way to change the data type of an existing array, is to make a copy of the array with the astype() method.

The astype() function creates a copy of the array, and allows you to specify the data type as a parameter.

The data type can be specified using a string, like 'f' for float, 'i' for integer etc. or you can use the data type directly like float for float and int for integer.

Example

Change data type from float to integer by using 'i' as parameter value:

```
import numpy as np
arr = np.array([1.1, 2.1, 3.1])
newarr = arr.astype('i')
print(newarr)
print(newarr.dtype)
```

Example
Change data type from float to integer by using int as parameter value:

```
import numpy as np
arr = np.array([1.1, 2.1, 3.1])
newarr = arr.astype(int)
print(newarr)
print(newarr.dtype)
```

Example
Change data type from integer to boolean:

```
import numpy as np
arr = np.array([1, 0, 3])
newarr = arr.astype(bool)
print(newarr)
print(newarr.dtype)
```

Lecture No. 36
Universal Function: Fast Element-Wise array Function

Universal functions in Numpy are simple mathematical functions. It is just a term that we gave to mathematical functions in the Numpy library. Numpy provides various universal functions that cover a wide variety of operations.

These functions include standard trigonometric functions, functions for arithmetic operations, handling complex numbers, statistical functions, etc. Universal functions have various characteristics which are as follows-

- These functions operates on **ndarray** (N-dimensional array) i.e Numpy's array class.
- It performs fast element-wise array operations.
- It supports various features like array broadcasting, type casting etc.
- Numpy, universal functions are objects those belongs to numpy.ufunc class.
- Python functions can also be created as a universal function using **frompyfunc** library function.
- Some **ufuncs** are called automatically when the corresponding arithmetic operator is used on arrays. For example when addition of two array is performed element-wise using '+' operator then np.add() is called internally.

Some of the basic universal functions in Numpy are-

Trigonometric functions:
These functions work on radians, so angles need to be converted to radians by multiplying by pi/180. Only then we can call trigonometric functions. They take an array as input arguments. It includes functions like-

Function	Description
sin, cos, tan	**compute sine, cosine and tangent of angles**
arcsin, arccos, arctan	**calculate inverse sine, cosine and tangent**
hypot	**calculate hypotenuse of given right triangle**
sinh, cosh, tanh	**compute hyperbolic sine, cosine and tangent**
arcsinh, arccosh, arctanh	**compute inverse hyperbolic sine, cosine and tangent**
deg2rad	**convert degree into radians**
rad2deg	**convert radians into degree**

Python code to demonstrate trigonometric function

```
import numpy as np

# create an array of angles
angles = np.array([0, 30, 45, 60, 90, 180])

# conversion of degree into radians
# using deg2rad function
radians = np.deg2rad(angles)

# sine of angles
print('Sine of angles in the array:')
sine_value = np.sin(radians)
print(np.sin(radians))

# inverse sine of sine values
print('Inverse Sine of sine values:')
print(np.rad2deg(np.arcsin(sine_value)))

# hyperbolic sine of angles
print('Sine hyperbolic of angles in the array:')
sineh_value = np.sinh(radians)
print(np.sinh(radians))

# inverse sine hyperbolic
print('Inverse Sine hyperbolic:')
print(np.sin(sineh_value))

# hypot function demonstration
base = 4
height = 3
print('hypotenuse of right triangle is:')
print(np.hypot(base, height))
```

Output:

```
Sine of angles in the array:
[ 0.00000000e+00  5.00000000e-01  7.07106781e-01  8.66025404e-01
  1.00000000e+00  1.22464680e-16]

Inverse Sine of sine values:
[ 0.00000000e+00  3.00000000e+01  4.50000000e+01  6.00000000e+01
  9.00000000e+01  7.01670930e-15]

Sine hyperbolic of angles in the array:
[ 0.          0.54785347  0.86867096  1.24936705  2.3012989
```

11.54873936]

Inverse Sine hyperbolic:
[0. 0.52085606 0.76347126 0.94878485 0.74483916 -0.85086591]

hypotenuse of right triangle is:
5.0

Statistical functions:
These functions are used to calculate mean, median, variance, minimum of array elements. It includes functions like-

Function	Description
amin, amax	**returns minimum or maximum of an array or along an axis**
ptp	**returns range of values (maximum-minimum) of an array or along an axis**
percentile(a, p, axis)	**calculate pth percentile of array or along specified axis**
median	**compute median of data along specified axis**
mean	**compute mean of data along specified axis**
std	**compute standard deviation of data along specified axis**
var	**compute variance of data along specified axis**
average	**compute average of data along specified axis**

-

Python code demonstrate statistical function
import numpy as np

construct a weight array
weight = np.array([50.7, 52.5, 50, 58, 55.63, 73.25,
49.5, 45])

minimum and maximum
print('Minimum and maximum weight of the students:
')
print(np.amin(weight), np.amax(weight))

range of weight i.e. max weight-min weight
print('Range of the weight of the students: ')

```
print(np.ptp(weight))

# percentile
print('Weight below which 70 % student fall: ')
print(np.percentile(weight, 70))

# mean
print('Mean weight of the students: ')
print(np.mean(weight))

# median
print('Median weight of the students: ')
print(np.median(weight))

# standard deviation
print('Standard deviation of weight of the students: ')
print(np.std(weight))

# variance
print('Variance of weight of the students: ')
print(np.var(weight))

# average
print('Average weight of the students: ')
print(np.average(weight))
```

Output:

```
Minimum and maximum weight of the students:
45.0 73.25

Range of the weight of the students:
28.25

Weight below which 70 % student fall:
55.317

Mean weight of the students:
54.3225

Median weight of the students:
51.6

Standard deviation of weight of the students:
```

8.05277397857

Variance of weight of the students:
64.84716875

Average weight of the students:
54.3225

Lecture No. 37

NumPy related Examples

Python program to demonstrate a need of NumPy

```
list1 = [1, 2, 3, 4 ,5, 6]
list2 = [10, 9, 8, 7, 6, 5]

# Multiplying both lists directly would give an
error.
print(list1*list2)
```

Output :
TypeError: can't multiply sequence by non-int of type 'list'

Python program to demonstrate the use of NumPy
arrays

```
import numpy as np

list1 = [1, 2, 3, 4, 5, 6]
list2 = [10, 9, 8, 7, 6, 5]

# Convert list1 into a NumPy array
a1 = np.array(list1)

# Convert list2 into a NumPy array
a2 = np.array(list2)

print(a1*a2)
```

Output :
array([10, 18, 24, 28, 30, 30])

Indexing using index arrays
Indexing can be done in numpy by using an array as an index. In case of slice, a view or shallow copy of the array is returned but in index array a copy of the original array is returned. Numpy arrays can be indexed with other arrays or any other sequence with the exception of tuples. The last element is indexed by -1 second last by -2 and so on.

Python program to demonstrate

```
# the use of index arrays.
import numpy as np

# Create a sequence of integers from 10 to 1 with a
step of -2
a = np.arange(10, 1, -2)
print("\n A sequential array with a negative step: \n",a)

# Indexes are specified inside the np.array method.
newarr = a[np.array([3, 1, 2 ])]
print("\n Elements at these indices are:\n",newarr)
```

Output :

```
A sequential array with a negative step:
[10  8  6  4  2]

Elements at these indices are:
[4 8 6]
```

-

```
import numpy as np

# NumPy array with elements from 1 to 9
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])

# Index values can be negative.
arr = x[np.array([1, 3, -3])]
print("\n Elements are : \n",arr)
```

Output :

```
Elements are:
[2 4 7]
```

Types of Indexing
There are two types of indexing :

1. Basic Slicing and indexing : Consider the syntax x[obj] where x is the array and obj is the index. Slice object is the index in case of basic slicing. Basic slicing occurs when obj is :

a slice object that is of the form start : stop : step

1. an integer
2. or a tuple of slice objects and integers

All arrays generated by basic slicing are always view of the original array.

- Python

```
# Python program for basic slicing.
import numpy as np

# Arrange elements from 0 to 19
a = np.arrange(20)
print("\n Array is:\n ",a)

# a[start:stop:step]
print("\n a[-8:17:1] = ",a[-8:17:1])

# The : operator means all elements till the end.
print("\n a[10:] = ",a[10:])
```

Output :

```
Array is:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]

a[-8:17:1]  =  [12 13 14 15 16]

a[10:] = [10 11 12 13 14 15 16 17 18 19]
```

Ellipsis can also be used along with basic slicing. Ellipsis (…) is the number of : objects needed to make a selection tuple of the same length as the dimensions of the array.

```
# Python program for indexing using
# basic slicing with ellipsis
import numpy as np

# A 3 dimensional array.
b = np.array([[[1, 2, 3],[4, 5, 6]],
          [[7, 8, 9],[10, 11, 12]]])

print(b[...,1]) #Equivalent to b[: ,: ,1 ]
```

Output :

```
[[ 2  5]
 [ 8 11]]
```

2. Advanced indexing : Advanced indexing is triggered when obj is :

- an ndarray of type integer or Boolean
- or a tuple with at least one sequence object
- is a non tuple sequence object

Advanced indexing returns a copy of data rather than a view of it. Advanced indexing is of two types integer and Boolean.

Purely integer indexing : When integers are used for indexing. Each element of first dimension is paired with the element of the second dimension. So the index of the elements in this case are (0,0),(1,0),(2,1) and the corresponding elements are selected.

-

```
# Python program showing advanced indexing
import numpy as np

a = np.array([[1 ,2 ],[3 ,4 ],[5 ,6 ]])
print(a[[0 ,1 ,2 ],[0 ,0 ,1]])
```

Output :
[1 3 6]

Boolean Indexing
This indexing has some boolean expression as the index. Those elements are returned which satisfy that Boolean expression. It is used for filtering the desired element values.

-

```
# You may wish to select numbers greater than 50
import numpy as np

a = np.array([10, 40, 80, 50, 100])
print(a[a>50])
```

Output :
[80 100]

-

```
# You may wish to square the multiples of 40
import numpy as np

a = np.array([10, 40, 80, 50, 100])
print(a[a%40==0]**2)
```

Output :
[1600 6400])

-

```
# You may wish to select those elements whose
# sum of row is a multiple of 10.
```

```
import numpy as np

b = np.array([[5, 5],[4, 5],[16, 4]])
sumrow = b.sum(-1)
print(b[sumrow%10==0])
```

Output :
array([[5, 5], [16, 4]])

Bit-twiddling functions:
These functions accept integer values as input arguments and perform bitwise operations on binary representations of those integers. It include functions like-

Function	Description
bitwise_and	performs bitwise and operation on two array elements
bitwies_or	performs bitwise or operation on two array elements
bitwise_xor	performs bitwise xor operation on two array elements
invert	performs bitwise inversion of an array elements
left_shift	shift the bits of elements to left
right_shift	shift the bits of elements to left

```
# Python code to demonstrate bitwise-function
import numpy as np

# construct an array of even and odd numbers
even = np.array([0, 2, 4, 6, 8, 16, 32])
odd = np.array([1, 3, 5, 7, 9, 17, 33])

# bitwise_and
print('bitwise_and of two arrays: ')
print(np.bitwise_and(even, odd))

# bitwise_or
print('bitwise_or of two arrays: ')
print(np.bitwise_or(even, odd))

# bitwise_xor
print('bitwise_xor of two arrays: ')
print(np.bitwise_xor(even, odd))
```

```
# invert or not
print('inversion of even no. array: ')
print(np.invert(even))

# left_shift
print('left_shift of even no. array: ')
print(np.left_shift(even, 1))

# right_shift
print('right_shift of even no. array: ')
print(np.right_shift(even, 1))
```

Output:

```
bitwise_and of two arrays:
[ 0  2  4  6  8 16 32]

bitwise_or of two arrays:
[ 1  3  5  7  9 17 33]

bitwise_xor of two arrays:
[1 1 1 1 1 1 1]

inversion of even no. array:
[ -1  -3  -5  -7  -9 -17 -33]

left_shift of even no. array:
[ 0  4  8 12 16 32 64]

right_shift of even no. array:
[ 0  1  2  3  4  8 16]
```

Lecture -38

Arithmetic with NumPy arrays- basic indexing and slicing- Boolean indexing- transposing array and Swapping Axes.

NumPy - Advanced Indexing

It is possible to make a selection from ndarray that is a non-tuple sequence, ndarray object of integer or Boolean data type, or a tuple with at least one item being a sequence object. Advanced indexing always returns a copy of the data. As against this, the slicing only presents a view.

There are two types of advanced indexing − **Integer** and **Boolean**.

Integer Indexing

This mechanism helps in selecting any arbitrary item in an array based on its Ndimensional index. Each integer array represents the number of indexes into that dimension. When the index consists of as many integer arrays as the dimensions of the target ndarray, it becomes straightforward.

In the following example, one element of specified column from each row of ndarray object is selected. Hence, the row index contains all row numbers, and the column index specifies the element to be selected.

Example

```
import numpy as np
x = np.array([[1, 2], [3, 4], [5, 6]])
y = x[[0,1,2], [0,1,0]]
print y
```

Its output would be as follows −

```
[1  4  5]
```

The selection includes elements at (0,0), (1,1) and (2,0) from the first array.

In the following example, elements placed at corners of a 4X3 array are selected. The row indices of selection are [0, 0] and [3,3] whereas the column indices are [0,2] and [0,2].

Example

```
import numpy as np
x = np.array([[ 0,  1,  2],[ 3,  4,  5],[ 6,  7,  8],[ 9, 10, 11]])
print 'Our array is:'
print x
print '\n'
```

```
rows = np.array([[0,0],[3,3]])
cols = np.array([[0,2],[0,2]])
y = x[rows,cols]

print 'The corner elements of this array are:'
print y
```

The output of this program is as follows −

```
Our array is:
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]

The corner elements of this array are:
[[ 0  2]
 [ 9 11]]
```

The resultant selection is an ndarray object containing corner elements.

Example

```
import numpy as np
x = np.array([[ 0,  1,  2],[ 3,  4,  5],[ 6,  7,  8],[ 9, 10, 11]])
print 'Our array is:'
print x
print '\n'

# slicing
z = x[1:4,1:3]

print 'After slicing, our array becomes:'
print z
print '\n'

# using advanced index for column
y = x[1:4,[1,2]]

print 'Slicing using advanced index for column:'
print y
```

The output of this program would be as follows −

```
Our array is:
[[ 0  1  2]
```

```
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

After slicing, our array becomes:

```
[[ 4  5]
 [ 7  8]
 [10 11]]
```

Slicing using advanced index for column:

```
[[ 4  5]
 [ 7  8]
 [10 11]]
```

Boolean Array Indexing

This type of advanced indexing is used when the resultant object is meant to be the result of Boolean operations, such as comparison operators.

Example

In this example, items greater than 5 are returned as a result of Boolean indexing.

```
import numpy as np
x = np.array([[ 0,  1,  2],[ 3,  4,  5],[ 6,  7,  8],[ 9, 10, 11]])
print 'Our array is:'
print x
print '\n'

# Now we will print the items greater than 5
print 'The items greater than 5 are:'
print x[x > 5]
```

The output of this program would be −

```
Our array is:
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]

The items greater than 5 are:
[ 6  7  8  9 10 11]
```

Example
In this example, NaN (Not a Number) elements are omitted by using ~ (complement operator).

```
import numpy as np
a = np.array([np.nan, 1,2,np.nan,3,4,5])
print a[~np.isnan(a)]
```

Its output would be −

```
[ 1.  2.  3.  4.  5.]
```

Example
The following example shows how to filter out the non-complex elements from an array.

```
import numpy as np
a = np.array([1, 2+6j, 5, 3.5+5j])
print a[np.iscomplex(a)]
```

Here, the output is as follows −

```
[2.0+6.j  3.5+5.j]
```

Lecture- 39
Mathematical and statistical methods- Sorting Unique and Other set Logic

numpy.unique() in Python

The numpy module of Python provides a function for finding unique elements in a numpy array. The numpy.unique() function finds the unique elements of an array and returns these unique elements as a sorted array. Apart from the unique elements, there are some optional outputs also, which are as follows:

- The output can be the indices of the input array which give the unique values
- The output can be the indices of the unique array which reconstruct the input array
- The output can be an array of the number of times each unique value comes in the input array.

Syntax

numpy.unique(a, return_index=False, return_inverse=False, return_counts=False, axis=None)
Parameters

These are the following parameters in numpy.mean() function:

a: array_like

This parameter defines the source array containing elements whose unique values are desired. The array will be flattened if it is not 1-D array.
Play Video

Return_index: bool(optional)

If this parameter is set True, the function will return the indices of the input array(along the specified axis if provided or in the flattened array), which results in the unique array.

return_inverse: bool(optional)

If this parameter is set True, the function will also return the indices of the input array(along the specified axis if provided or in the flattened array), which can be used to reconstruct the input array.

Return_counts: bool(optional)

If this parameter is set True, the function will return the number of times each unique item appeared in the input array 'a'.

axis: int or None(optional)

This parameter defines the axis to operate on. If this parameter is not set, then the array 'a' will be flattened. If this parameter is an integer, then the subarrays indexed by the given axis will be flattened and treated as an element of a 1-D array with the dimension of the given axis. Structured arrays or object arrays that contain objects are not supported if the axis 'kwarg' is used.

Returns
This function returns four types of outputs which are as follows:

unique: ndarray
In this output, a ndarray will be shown that contain sorted unique values.

unique_indices: ndarray(optional)
In this output, a ndarray will be shown that contains the indices of the first occurrences of the unique values in the original array. This output is only provided if return_index is True.

unique_inverse: ndarray(optional)
In this output, a ndarray will be shown that contains the indices to reconstruct the original array from the unique array. This output is only provided if return_inverse is True.

unique_counts: ndarray(optional)
In this output, a ndarray will be shown that contains the number of times each of the unique values comes up in the original array. This output is only provided if return_counts is True.

Example

```
import numpy as np
a=np.unique([1,2,3,4,3,6,2,4])
a
```

Output:

```
array([1, 2, 3, 4, 6])
```

In the above code

We have imported numpy with alias name np.
We have declared the variable 'a' and assigned the returned value of np.unique() function.
We have passed the number of elements in the function.
Lastly, we tried to print the value of 'a'.
In the output, a ndarray has been shown, which contains unique elements.
Example 2:

```
a=np.array([[1,2,2,3,9],[1,4,3,5,8]])
a
b=np.unique(a)
b
```

Output:

```
array([[1, 2, 2, 3, 9],
       [1, 4, 3, 5, 8]])
array([1, 2, 3, 4, 5, 8, 9])
```

Example 3:

```
import numpy as np
a = np.array([[1, 1, 0], [1, 1, 0], [2, 3, 4],[5, 9, 8],[2, 3, 4]])
a
b=np.unique(a, axis=0)
b
```

Output:

```
array([[1, 1, 0],
       [1, 1, 0],
       [2, 3, 4],
       [5, 9, 8],
       [2, 3, 4]])
array([[1, 1, 0],
       [2, 3, 4],
       [5, 9, 8]])
```

In the above code

- We have imported numpy with alias name np.
- We have created a multi-dimensional array 'a'.
- We have declared the variable 'b' and assigned the returned value of np.unique() function.
- We have passed the multi-dimensional array 'a' and axis as 0 in the function.
- Lastly, we tried to print the value of 'b'.

In the output, a ndarray has been shown that contains unique rows of the source array 'a'.

Example

```
import numpy as np
a = np.array([[1, 1, 0], [1, 1, 0], [2, 2, 4],[5, 5, 8],[2, 2, 4]])
a
b=np.unique(a, axis=1)
b
```

Output:

```
array([[1, 1, 0],
       [1, 1, 0],
       [2, 2, 4],
       [5, 5, 8],
       [2, 2, 4]])
array([[0, 1],
       [0, 1],
       [4, 2],
       [8, 5],
       [4, 2]])
```

Note: When we set axis as 1 then this function returns the unique columns from the source array.

Example 5: Use return_index

```
import numpy as np
a = np.array(['d', 'b', 'b', 'z', 'a'])
result, indices=np.unique(a,return_index=True)
result
indices
a[indices]
```

Output:
array(['a', 'b', 'd', 'z'], dtype='|S1')
array([4, 1, 0, 3], dtype=int64)
array(['a', 'b', 'd', 'z'], dtype='|S1')

In the above code

- We have imported numpy with alias name np.
- We have created an array 'a'.
- We have declared the variable 'result' and 'indices' and assigned the returned value of np.unique() function.
- We have passed the array 'a' and set return_index to True in the function.
- Lastly, we tried to print the value of 'result', 'indices', and array elements, which indicates the indices('a [indices]').

In the output, a ndarray has been shown that contains the indices of the original array that give unique values.

Example : Use return_inverse
We can reconstruct the input array from the unique values in the following way:

```
import numpy as np
a = np.array([1, 2, 6, 4, 5, 3, 2])
result, indices=np.unique(a,return_inverse=True)
result
indices
a[indices]
```

Output:
array([1, 2, 3, 4, 5, 6])
array([0, 1, 5, 3, 4, 2, 1], dtype=int64)
array([1, 2, 3, 4, 5, 6, 2])

Python set operation.
In mathematics, a set is a collection of distinct objects, considered as an object in its own right. For example, the numbers 2, 4, and 6 are distinct objects when considered separately, but when they are considered collectively they form a single set of size three, written {2,4,6}.
Operations on Sets

Operation	Notation	Meaning
Intersection	A ∩ B	all elements which are in both and
Union	A ∪ B	all elements which are in either or (or both)
Difference	A − B	all elements which are in but not in

Complement	(or)	all elements which are not in

In python, compared to list, the main advantages of using a set are that it has optimized functions for checking whether a specific element is a member of the set or not. This is based on a hash table data structure.
Methods for Sets

add(x) Method: It adds the item x to a set if it is non-preexisting.

```
A = {"AA", "BB", "CC"}
A.add("VV")
```

This will add VV in A set.

union(s) Method: It returns a union of two set.Using the operator '|'between 2 existing sets is the same as writing My_Set1.union(My_Set2)..

```
A = {"AA", "BB", "CC"}
B = {"MM", "NN"}
Z = A.union(B)
OR
Z = A|B
```

Set population set will have components of both A and B.

intersect(s) Method: It returns an intersection of two givensets. The '&' operator can be used in this operation.

```
S = A.intersection(B)
```

Set victims will contain the common elements of A and B.

difference(s) Method: Returns a set containing all the elements which are existing in the first set but not present in the second set. We can use '-' operator here.

```
W = A.difference(B)
OR
S = A – B
```

Set safe will have all the elements that are in A but not in B.

clear() Method: The whole existing set will become empty.

```
B.clear()
```

Clears B set

Operators for Sets

Sets and frozen sets support the following operators -

```
key in s        # containment check
key not in s    # non-containment check
s1 == s2        # s1 is equivalent to s2
s1 != s2        # s1 is not equivalent to s2
```

s1 <= s2 # s1is subset of s2 s1 < s2 # s1 is proper subset of s2 s1 >= s2
s1is superset of s2
s1 > s2 # s1 is proper superset of s2
s1 | s2 # the union of s1 and s2
s1 & s2 # the intersection of s1 and s2
s1 – s2 # the set of elements in s1 but not s2
s1 ^ s2 # the set of elements in precisely one of s1 or s2

Example

```
# Python program to demonstrate working# of
# Set in Python
# creating two sets
My_Set1 = set()
My_Set2 = set()
# Adding elements to My_Set1

for i in range(1, 6):
  My_Set1.add(i)

# Adding elements to My_Set2
for i in range(3, 8):
  My_Set2.add(i)
print("My_Set1 = ", My_Set1)
print("My_Set2 = ", My_Set2)
print("\n")

# Union of My_Set1 and My_Set2
My_Set3 = My_Set1 | My_Set2# My_Set1.union(My_Set2)
print("Union of My_Set1&My_Set2: My_Set3 = ", My_Set3)

# Intersection of My_Set1 and My_Set2
My_Set4 = My_Set1&My_Set2# My_Set1.intersection(My_Set2)
print("Intersection of My_Set1&My_Set2: My_Set4 = ", My_Set4)
print("\n")

# Checking relation between My_Set3 and My_Set4
if My_Set3>My_Set4: # My_Set3.issuperset(My_Set4)
  print("My_Set3 is superset of My_Set4")
elif My_Set3<My_Set4: # My_Set3.issubset(My_Set4)
  print("My_Set3 is subset of My_Set4")
else : # My_Set3 == My_Set4
  print("My_Set3 is same as My_Set4")

# displaying relation between My_Set4 and My_Set3
```

```
if My_Set4<My_Set3: # My_Set4.issubset(My_Set3)
   print("My_Set4 is subset of My_Set3")
   print("\n")

# difference between My_Set3 and My_Set4
My_Set5 = My_Set3 - My_Set4
print("Elements in My_Set3 and not in My_Set4: My_Set5 = ", My_Set5)
print("\n")

# check if My_Set4 and My_Set5 are disjoint sets
if My_Set4.isdisjoint(My_Set5):
   print("My_Set4 and My_Set5 have nothing in common\n")

# Removing all the values of My_Set5
My_Set5.clear()
print("After applying clear on sets My_Set5: ")
print("My_Set5 = ", My_Set5)
```

Output

```
My_Set1 = {1, 2, 3, 4, 5}
My_Set2 = {3, 4, 5, 6, 7}
Union of My_Set1&My_Set2: My_Set3 = {1, 2, 3, 4, 5, 6, 7}
Intersection of My_Set1&My_Set2: My_Set4 = {3, 4, 5}
My_Set3 is superset of My_Set4
My_Set4 is subset of My_Set3
Elements in My_Set3 and not in My_Set4: My_Set5 = {1, 2, 6, 7}
My_Set4 and My_Set5 have nothing in common
After applying clear on sets My_Set5:
My_Set5 = set()
```

Lecture 40
Data Visualization

Packages of Data Visualization in Python

It can be hard to fully comprehend what users see in a tabular format when handling a huge amount of data sets. We must visualize or represent our data in pictorial form to better understand what it conveys and make better it systematic and select appropriate models for it. This allows us to see trends, correlations, and dynamics that we wouldn't see in a chart or CSV file.

Data visualization is a branch of data analysis that focuses on visualizing data. It plots data graphically and is a good way to communicate data inferences.
The human brain has an easy and fast processing time and understanding of data when presented in pictures, maps, and graphs. We can always get a visual description of our data by using data visualization. Data visualization is important in representing both big and small data sets. However, it is especially useful when dealing with large data sets where it is impossible to see all our records, process, and understand them.

Data Visualization in Python

In today's world, data visualization in Python is probably one of the most widely used functionalities for data science with Python. Python libraries include various features that allow users to create highly customized, classy, and interactive plots. Python includes several plotting libraries, such as Matplotlib, Seaborn, and other data visualization packages. Each has its own set of features for constructing informative, customized, and intriguing plots to present information most simply and effectively possible.
Python has several libraries for visualizing data, each with its features. Four of these libraries will be discussed in this tutorial. Each of these libraries has its own set of functionalities and can support a variety of graph types.

- Matplotlib
- Seaborn
- Bokeh
- Plotly

Useful Python Packages for Visualizations

Matplotlib

Matplotlib is a Python visualization library for 2D array plots. Matplotlib is a Python library that uses the NumPy library. It works with Python and IPython shells, as well as Jupyter notebooks and web application server software. Matplotlib includes a wide range of plots, such as scatter, line, bar, histogram, and others, that can assist us in delving deeper into trends, behavioral patterns, and correlations. John Hunter first launched it in 2002.

Scatter Plot

Scatter plots can be used to visualize relationships among variables, with dots representing the relation. A scatter plot is created using the matplotlib library's scatter() method.

```
import pandas as pd
import matplotlib.pyplot as plt

# reading the csv data set
dataset = pd.read_csv("tips.csv")

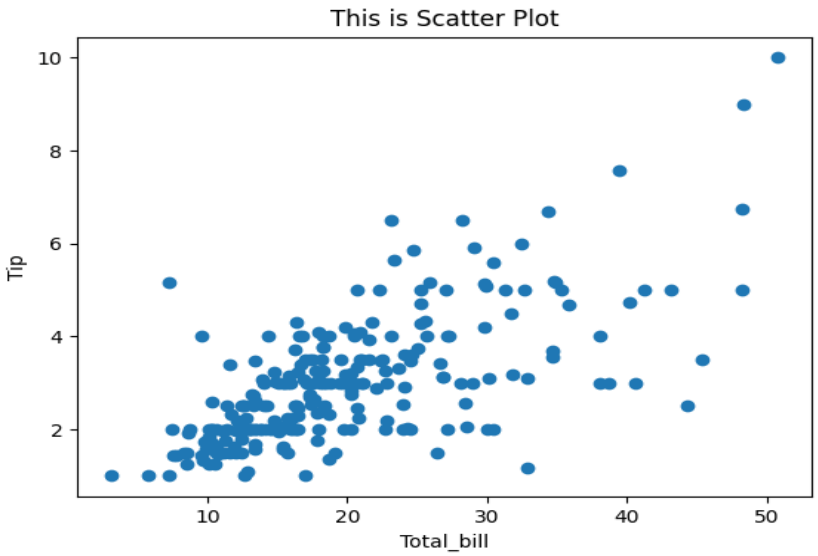
# Plotting Scatter plot of total_bill vs tip
plt.scatter(dataset['total_bill'], dataset['tip'])

# Giving our plot a title
plt.title("This is Scatter Plot")

# GIving x and y labels names
plt.xlabel('Total_bill')
plt.ylabel('Tip')

plt.show()
```

Output:



Bar Chart

A bar plot or a bar chart is just a graph that uses rectangular bars which have lengths and heights proportional to the data values they represent to represent a category of data.

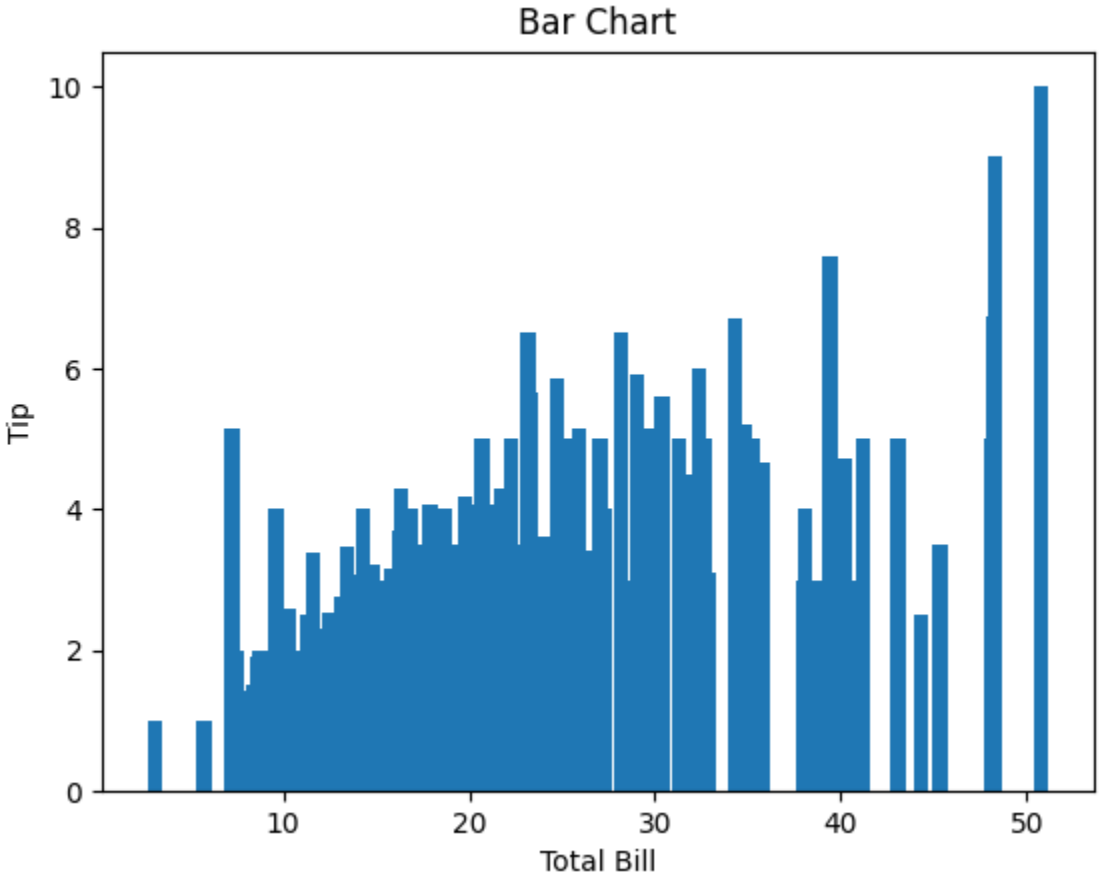
```
import pandas as pd
import matplotlib.pyplot as plt
```

```
# reading the csv data set
dataset = pd.read_csv("tips.csv")

# Plotting Scatter plot of total_bill vs tip
plt.bar(dataset['total_bill'], dataset['tip'])
# Giving our plot a title
plt.title("Bar Chart")

# GIving x and y labels names
plt.xlabel('Total Bill')
plt.ylabel('Tip')
```

1. plt.show()
   Output:



Seaborn

Seaborn is a Python library for creating statistical representations based on datasets. It is built on top of matplotlib and is used to create various visualizations. It's built on top of pandas' data structures. The library conducts the necessary modeling and aggregation internally to create insightful visuals.

Line Plot

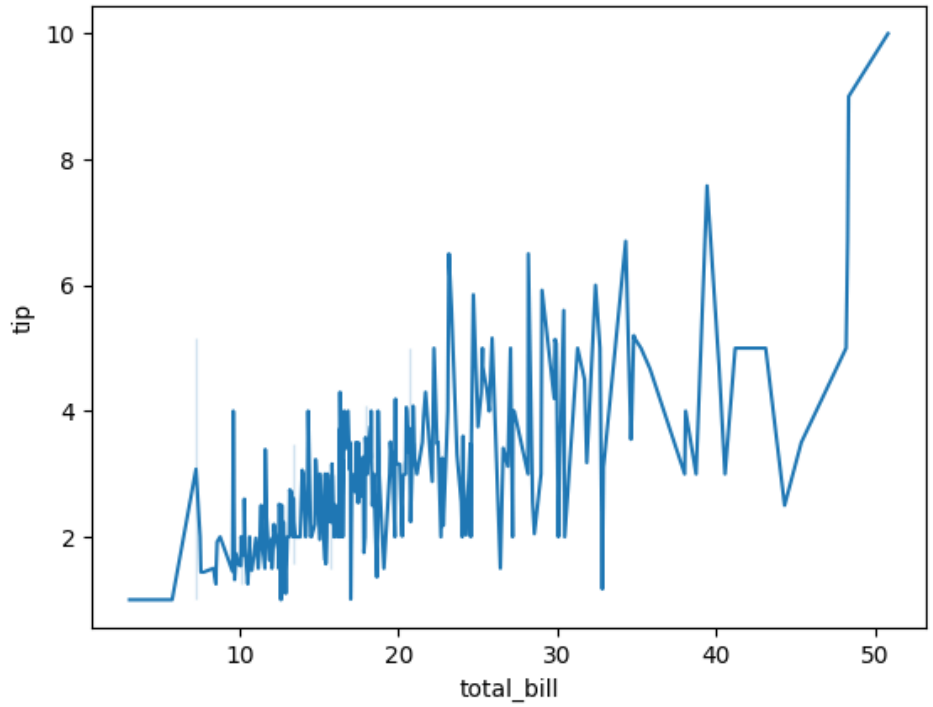
The lineplot() method was used to plot a line plot in Seaborn. We can also pass just the data argument in this case.

```
# importing the required packages
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt


# reading the csv data set using pandas
dataset = pd.read_csv("tips.csv")

sns.lineplot(x='total_bill', y='tip', data=dataset)
plt.show()
```

Output:



Bokeh

Bokeh is a modern web browser-based interactive visualization library. It can create engaging plots and dynamic dashboards with huge or streaming data assets. The library contains many intuitive graphs that can be used to create solutions. It has a close relationship with the PyData tools. The library is ideal for creating customized visuals based on specific use cases. The visual effects can also be crafted interactive to support a what-if scenario model.

Line Chart

The plotting and scheming module's line() method can also be used to create a line plot.

```
# importing the required modules
import bokeh.plotting
import pandas as pd

# creating an object for the figure
chart = bokeh.plotting.figure(title="Bokeh Bar Chart")

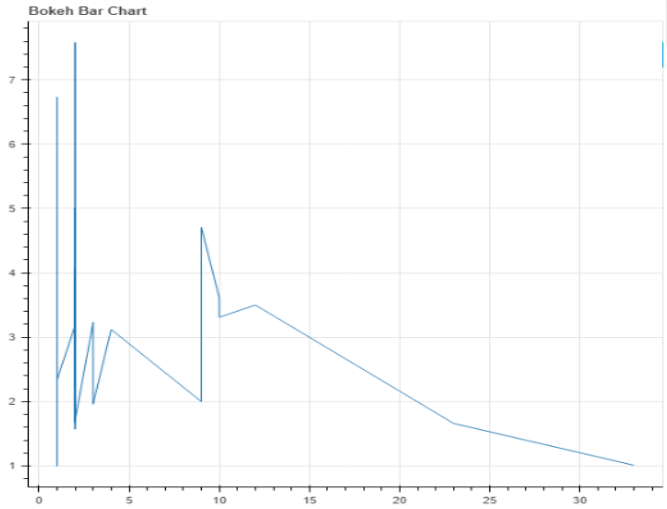
# reading the csv dataset through pandas
dataset = pd.read_csv("tips.csv")

# Counting unique columns of thwe tips file
count = dataset['tip'].value_counts()

# plotting the graph
chart.line(count, dataset['tip'])

# showing the figure
bokeh.plotting.show(chart)
```

Output:



Adding Widgets

- Buttons, sliders, checkboxes, and other GUI elements similar to those found in HTML forms are available in Bokeh. These include an interactive user interface to the graph that enables us to change plot parameters, modify plot data, etc. Let's look at how to utilize and add some of the most common widgets.
- Buttons: Adds a normal button item to the plot with this widget. The CustomJS() function of the models class must be passed a custom JavaScript function.
- CheckboxGroup: A conventional check box is added to the plot. Similarly to buttons, we must pass the custom JavaScript method to the models class's CustomJS() method.

- RadioGroup: This component adds a basic radio button as well as allows us to use a personalized JavaScript function.

plotly

plotly.py is a Python visualization library that is interactive, accessible, high-level, declarative, and browser-based. Scientific graphs, 3D charts, statistical plots, and financial charts are just a few visualizations available. Plotly charts can be construed in Jupyter notebooks, as standalone Html documents, or as live web pages. Interaction and editing options are available in the Plotly library. Both locally and in a web browser, the robust API performs admirably.

Scatter Plot

The scatter() function of plotly.express can be used to create a scatter plot in Plotly. An additional data argument too is necessary here, as it is for Seaborn.

```
# importing the required modules
import pandas as pd
import plotly.express


# reading the csv dataset through pandas
dataset = pd.read_csv("tips.csv")

# plotting our scatter plot
graph = plotly.express.scatter(dataset, x="total_bill", y="tip", color='smoker')

# displaying the plot created
graph.show()
```

Output:

